

HOSPITAL RESOURCE ALLOCATOR

PROJECT DETAILES

Project Title : Hospital Resource Allocator [Greedy]

Number : 15

GROUP MEMBERS

Name	ID	Department	Main Task
Omar Sayeed El-Said Abady	2403258221	AI(G1)	Report
Akram Mohamed Sayeed	2403254754	AI(G1)	Greedy
Aisha Alaa El-Den Ahmed	2403256725	AI(G1)	Simulation
Ahmed Gamal Darwish	2403251323	AI(G1)	Data Set && Sorting
Ahmed Shafik Ahmed	2403253954	AI(G1)	GUI

OVERVIEW

1. Data Set

- **id**: a unique identifier for each patient
- **urgency**: the severity level of the case, from **1 (lowest)** to **5 (highest)**
- **type**: the specific hospital resource required by the patient (**Doctor / Room / Equipment**)
- **arrival**: the time in minutes when the patient arrives and becomes available for scheduling

2. Sorting

This function uses the **Bubble Sort** algorithm with a small optimization. The optimization is the flag variable, which helps stop the algorithm early if the list becomes sorted before all passes are completed.

3. Resources

The resources dictionary stores the number of available doctors, rooms, and equipment units in the hospital.

The **resource_free** dictionary keeps track of when each resource becomes available.

At the start of the simulation, all values are set to zero, meaning that every resource is initially free.

4. Greedy Best First Search

The **GBFS_hospital** function is responsible for scheduling patients and assigning them to available hospital resources.

It gives priority to patients with higher urgency after they arrive, then assigns them to the earliest free doctor, room, or equipment unit depending on their required resource type.

The function also records the treatment schedule, calculates waiting times, measures resource usage, and identifies any patients who could not be treated within the 120-minute time limit.

5. Simulation

This section prints the final results of the hospital scheduling algorithm.

It shows the treatment schedule for all assigned patients, including their IDs, resource types, and treatment times.

It also displays the number of untreated patients, the average waiting time for each urgency level, and the usage percentage of doctors, rooms, and equipment.

These results make it easier to evaluate how effectively the algorithm allocates hospital resources during the 120-minute simulation.

6. GUI

This section creates a **Tkinter-based** graphical user interface for the hospital simulation system.

The interface includes a button to run the simulation, a table to display the patient allocation schedule, and a text area to show summary results such as average waiting time, resource usage, and untreated patients.

The table rows are colour-coded by urgency level, which makes the output easier to read and analyze.

This GUI improves the presentation of the simulation results and makes the system more interactive and user-friendly.

1 Data Set :

```
patients = [  
  {"id": 121, "urgency": 5, "type": "Doctor", "arrival": 1},  
  {"id": 132, "urgency": 3, "type": "Room", "arrival": 2},  
  {"id": 136, "urgency": 4, "type": "Equipment", "arrival": 1},  
  {"id": 140, "urgency": 2, "type": "Doctor", "arrival": 4},  
  {"id": 159, "urgency": 1, "type": "Room", "arrival": 5},  
  {"id": 162, "urgency": 5, "type": "Equipment", "arrival": 3},  
  {"id": 171, "urgency": 4, "type": "Doctor", "arrival": 2},  
  {"id": 187, "urgency": 3, "type": "Room", "arrival": 2},  
  {"id": 190, "urgency": 2, "type": "Equipment", "arrival": 2},  
  {"id": 101, "urgency": 5, "type": "Doctor", "arrival": 5},  
  {"id": 255, "urgency": 5, "type": "Doctor", "arrival": 1},  
  {"id": 298, "urgency": 3, "type": "Room", "arrival": 2},  
  {"id": 245, "urgency": 4, "type": "Equipment", "arrival": 1},  
  {"id": 256, "urgency": 2, "type": "Doctor", "arrival": 4},  
  {"id": 213, "urgency": 1, "type": "Room", "arrival": 5},  
  {"id": 241, "urgency": 5, "type": "Equipment", "arrival": 9},  
  {"id": 233, "urgency": 4, "type": "Doctor", "arrival": 2},  
  {"id": 212, "urgency": 3, "type": "Room", "arrival": 2},  
  {"id": 211, "urgency": 2, "type": "Equipment", "arrival": 2},  
  {"id": 222, "urgency": 5, "type": "Doctor", "arrival": 5},  
  {"id": 111, "urgency": 1, "type": "Room", "arrival": 2},  
  {"id": 122, "urgency": 4, "type": "Equipment", "arrival": 1},  
  {"id": 130, "urgency": 3, "type": "Doctor", "arrival": 4},  
  {"id": 145, "urgency": 2, "type": "Room", "arrival": 7},  
  {"id": 152, "urgency": 5, "type": "Equipment", "arrival": 4},  
  {"id": 161, "urgency": 4, "type": "Doctor", "arrival": 2},  
  {"id": 177, "urgency": 3, "type": "Room", "arrival": 3},  
  {"id": 185, "urgency": 2, "type": "Equipment", "arrival": 4},  
  {"id": 119, "urgency": 1, "type": "Doctor", "arrival": 9},  
  {"id": 120, "urgency": 5, "type": "Room", "arrival": 8}  
]
```

The dataset used in this project consists of 60 hardcoded patient records. Each patient record contains four main attributes: patient ID, urgency level, required resource type, and arrival time.

The patient ID is a unique value used to distinguish each patient in the system.

The urgency level ranges from 1 to 5, where 5 represents the most critical cases and 1 represents the least urgent cases.

The resource type indicates whether the patient requires a Doctor, a Room, or Equipment.

The arrival time shows when the patient enters the system during the simulation period.

This dataset includes a variety of urgency levels and resource types to create a realistic hospital scheduling scenario.

The difference in arrival times allows the algorithm to simulate waiting queues and resource competition.

Such variation helps evaluate how effectively the greedy algorithm prioritizes critical patients and allocates available resources.

Overall, this dataset provides a practical and consistent basis for testing hospital scheduling performance during the 120-minute simulation.

2 Resources :



```
resources = {"Doctor": 5, "Room": 4, "Equipment": 3}
resource_free = {
    "Doctor": [0] * 5,
    "Room": [0] * 4,
    "Equipment": [0] * 3
}
```

The resources dictionary defines the number of available doctors, rooms, and equipment units in the hospital.

It shows that the system contains 5 doctors, 4 rooms, and 3 equipment units.

The resource_free dictionary is used to track when each individual resource becomes available.

All values are initially set to zero, which means that every resource is free at the start of the simulation.

During execution, these values are updated whenever a resource is assigned to a patient, helping the algorithm manage resource allocation correctly.

3 Sorting :

```
def sort_patients(patients):
    n=len(patients)
    for i in range(n):
        flag = True
        for j in range(n-i-1):
            if patients[j]["arrival"] > patients[j+1]["arrival"]:
                temp=patients[j]
                patients[j] = patients[j+1]
                patients[j+1] = temp
                flag=False
            elif patients[j]["arrival"] == patients[j+1]["arrival"]:
                if patients[j]["urgency"] < patients[j + 1]["urgency"]:
                    temp = patients[j]
                    patients[j] = patients[j + 1]
                    patients[j + 1] = temp
                    flag=False
        if flag:
            break
    sort_patients(patients)
```

The `sort_patients()` function is used to organize the patient list before the resource allocation process begins.

Its main purpose is to ensure that patients are arranged in a logical order that supports fair and efficient scheduling.

The function sorts patients primarily by **arrival time in ascending order**, so that patients who arrive earlier are considered first.

If two patients have the same arrival time, the function then compares their **urgency levels**.

In this case, the patient with the **higher urgency value** is placed before the other patient.

This sorting rule helps the system respect both the order of arrival and the medical priority of critical cases.

The function is implemented using the **Bubble Sort algorithm**, which repeatedly compares adjacent elements and swaps them when necessary.

It also uses a flag variable to detect whether any swaps occurred during a pass.

If no swaps are made, the algorithm stops early, which improves efficiency when the list is already sorted.

Overall, this preprocessing step provides a well-structured input for the scheduling algorithm and improves the accuracy of patient allocation.

4 Greedy Best First Search :

```
def GBFS_hospital(patients):
    sort_patients(patients)
    schedule = []
    wait_times = {i: [] for i in range(1, 6)}
    total_busy_time = {"Doctor": 0, "Room": 0, "Equipment": 0}
    not_tread = []
    treated = [False] * len(patients)
    processed_count = 0
    current_time = 0
    while processed_count < len(patients):
        available = [(i, p) for i, p in enumerate(patients)
                     if not treated[i] and p['arrival'] <= current_time]
        if not available:
            next_arrivals = [p['arrival'] for i, p in enumerate(patients) if not treated[i]]
            if next_arrivals:
                current_time = min(next_arrivals)
                continue
            else:
                break
        target_idx, patient = max(available, key=lambda x: (x[1]['urgency'], -x[1]
        ['arrival']))
        patient_type = patient["type"]
        resource_free[patient_type].sort()
        early_free = resource_free[patient_type][0]
        start_time = max(patient['arrival'], early_free)
        if start_time > 120:
            not_tread.append(patient['id'])
            treated[target_idx] = True
            processed_count += 1
            continue
        end_time = start_time + 10
        wait_time = start_time - patient['arrival']
        resource_free[patient_type][0] = end_time
        treated[target_idx] = True
        processed_count += 1
        schedule.append({
            "ID": patient['id'], "Type": patient_type,
            "Start": start_time, "End": end_time,
            "Urgency": patient['urgency']
        })
        wait_times[patient['urgency']].append(wait_time)
        total_busy_time[patient_type] += 10
        current_time = min(min(v) for v in resource_free.values())
    return schedule, wait_times, total_busy_time, not_tread
```

The GBFS_hospital() function is the main part of the program responsible for scheduling patients and allocating hospital resources.

It begins by sorting the patients based on arrival time and urgency to prepare the data for processing.

Then, it initializes the schedule list, waiting-time records, total busy time for each resource type, and a list of untreated patients.

The function also creates a treated list to track which patients have already been assigned to treatment.

During execution, it repeatedly checks which patients have already arrived and are still waiting. From the available patients, it selects the one with the highest urgency as the next patient to be treated.

After that, it finds the earliest available resource of the required type, such as a doctor, room, or equipment unit.

If the patient can be treated within the 120-minute limit, the function records the start time, end time, and waiting time in the schedule.

It also updates the resource availability and adds the treatment duration to the total busy time of that resource type.

Finally, the function returns the full treatment schedule, waiting times, resource usage, and the list of patients who could not be treated during the simulation.

5 Simulation :

```
results, waits, usage, ignored = GBFS_hospital(patients)
print(f"{'Patient ID':<10} | {'Type':<10} | {'Start':<6} | {'End':<6}")
print("-" * 55)
for r in results:
    print(f"{r['ID']:<10} | {r['Type']:<10} | {r['Start']:<6} | {r['End']:<6}")
untread_count = len(ignored)
print(f"\nTotal patients untread = {untread_count}")
if untread_count > 0:
    print(f"Untreated Patient IDs: {ignored}")

print("\n--- Average Wait Time ---")
for urg in range(5, 0, -1):
    avg = sum(waits[urg])/len(waits[urg]) if waits[urg] else 0
    print(f"Urgency {urg}: {avg:.1f} mins")

print("\n--- Resource Usage ---")
for r_type, busy in usage.items():
    total_available = 120 * resources[r_type]
    percent = (busy / total_available) * 100
    print(f"{r_type}: {percent:.1f}%")
```

The GBFS_hospital(patients) function is called to execute the hospital scheduling process and generate the final results.

It returns four outputs: the treatment schedule, waiting times by urgency level, resource usage, and the list of untreated patients.

The program first prints a formatted table showing each treated patient's ID, resource type, start time, and end time.

This table represents the final assignment schedule produced by the algorithm during the simulation.

After that, the program calculates the total number of patients who were not treated within the available time period.

If there are untreated patients, their IDs are displayed to make the result clear and easy to analyze.

The next part of the code computes the average waiting time for each urgency level from 5 down to 1.

This helps evaluate whether patients with higher urgency received faster treatment than less

critical cases.

Finally, the program calculates the utilisation percentage of each resource type by comparing busy time with total available time.

Overall, this section presents the final output of the simulation and helps measure the performance and efficiency of the scheduling algorithm.

6

GUI :

```

import tkinter as tk
from tkinter import ttk
def run_simulation():
    results, waits, usage, ignored = GBFS_hospital(patients)
    for row in tree.get_children():
        tree.delete(row)
    for r in results:
        tag = f"urg{r['Urgency']}"
        tree.insert("", "end", values=(r["ID"], r["Type"], r["Start"], r["End"], r["Urgency"]), tags=
(tag,))
    output_text.delete("1.0", tk.END)
    output_text.insert(tk.END, "=== Average Wait Time ===\n")
    for urg in range(5, 0, -1):
        avg = sum(waits[urg])/len(waits[urg]) if waits[urg] else 0
        output_text.insert(tk.END, f"Urgency {urg}: {avg:.1f} mins\n")

    output_text.insert(tk.END, "\n=== Resource Usage ===\n")
    for r_type, busy in usage.items():
        total_available = 120 * resources[r_type]
        percent = (busy / total_available) * 100
        output_text.insert(tk.END, f"{r_type}: {percent:.1f}%\n")

    output_text.insert(tk.END, f"\n=== Untreated Patients ===\n")
    output_text.insert(tk.END, f"Count: {len(ignored)}\nIDs: {ignored}")
    root = tk.Tk()
    root.title(" Hospital Resource Allocator")
    root.geometry("1500x1400")
    root.configure(bg="#f0f4f7")
    title = tk.Label(root, text="Hospital Resource Allocator",
        font=("Arial", 18, "bold"), bg="#f0f4f7", fg="#2c3e50")
    title.pack(pady=10)
    btn = tk.Button(root, text="▶ Run Simulation", command=run_simulation,
        font=("Arial", 12, "bold"),
        bg="#3498db", fg="white",
        padx=10, pady=5)
    btn.pack(pady=10)
    frame = tk.Frame(root)
    frame.pack()
    columns = ("ID", "Type", "Start", "End", "Urgency")
    tree = ttk.Treeview(frame, columns=columns, show="headings", height=15)
    for col in columns:
        tree.heading(col, text=col)
        tree.column(col, anchor="center", width=100)
    tree.pack(pady=10, padx=20, fill="both", expand=True)
    tree.tag_configure("urg5", background="#ff6b6b")
    tree.tag_configure("urg4", background="#ffa94d")
    tree.tag_configure("urg3", background="#fff176")
    tree.tag_configure("urg2", background="#69db7c")
    tree.tag_configure("urg1", background="#74c0fc")
    output_text = tk.Text(root, height=15, width=90, font=("Consolas", 10))
    output_text.pack(pady=10, padx=10)
    root.mainloop()

```

This part of the program creates a graphical user interface (GUI) using Tkinter to display the hospital simulation results.

It includes a button that allows the user to run the simulation and generate the allocation

output.

The results are shown in a table using Treeview, where each patient's ID, type, start time, end time, and urgency are displayed.

A text area is also used to present summary information such as average waiting time, resource usage, and untreated patients.

In addition, different row colors are used to represent urgency levels, making the output easier to read and analyze.

7 OUT PUT :

Patient ID	Type	Start	End

121	Doctor	1	11
255	Doctor	1	11
136	Equipment	1	11
245	Equipment	1	11
122	Equipment	1	11
422	Equipment	11	21
102	Doctor	1	11
137	Equipment	11	21
802	Doctor	1	11
537	Equipment	11	21
628	Doctor	1	11
709	Room	1	11
171	Doctor	11	21
233	Doctor	11	21
161	Doctor	11	21
106	Room	2	12
561	Doctor	11	21
506	Room	2	12
132	Room	2	12
162	Equipment	21	31
152	Equipment	21	31
125	Doctor	11	21
352	Equipment	21	31
625	Doctor	21	31
101	Doctor	21	31
222	Doctor	21	31
112	Equipment	31	41
812	Equipment	31	41
120	Room	11	21
720	Room	12	22

241	Equipment	31	41
123	Equipment	41	51
923	Equipment	41	51
51	61		
145	Room	32	42
645	Room	32	42
128	Doctor	31	41
111	Room	32	42
311	Room	41	51
159	Room	42	52
213	Room	42	52
124	Equipment	61	71
624	Equipment	61	71
119	Doctor	31	41
719	Doctor	31	41
109	Room	42	52

Total patients untread = 0

--- Average Wait Time ---

Urgency 5: 13.4 mins

Urgency 4: 9.8 mins

Urgency 3: 10.9 mins

Urgency 2: 29.3 mins

Urgency 1: 32.8 mins

--- Resource Usage ---Doctor: 33.3%

Room: 41.7%

Equipment: 55.6%

CONCLUSION

This project presents a **Hospital Resource Allocator** based on a **greedy scheduling approach** to manage limited hospital resources efficiently. The system uses a fixed dataset of **60 patients**, where each patient has a unique ID, an urgency level from **1 to 5**, a required resource type (**Doctor, Room, or Equipment**), and an arrival time. Before allocation, patients are sorted by arrival time and urgency to organize the input data. The hospital resources are limited to **5 doctors, 4 rooms, and 3 equipment units**, and each treatment requires a fixed **10-minute time slot**. During the simulation, the algorithm repeatedly selects the highest-priority available patient and assigns them to the earliest free resource of the required type. The program then generates a full treatment schedule, calculates the average waiting time for each urgency level, measures resource utilisation, and identifies untreated patients within the **120-minute simulation period**. In addition, a **Tkinter-based GUI** is implemented to display the schedule and results in a clear and interactive way. Overall, the project demonstrates how greedy allocation can be used to improve hospital scheduling, reduce waiting time, and make better use of available resources.